BIBLIOTECAS PYTHON - CONTINUAÇÃO

- Silvana Philippi Camboim - silvanacamboim@gmail.com

- Programa de pós-graduação em Ciências Geodésicas/Programa de pós-graduação em Planejamento Urbano/ Curso de Engenharia Cartográfica e de Agrimensura

UFPR
UNIVERSIDADE FEDERAL DO PARANÁ

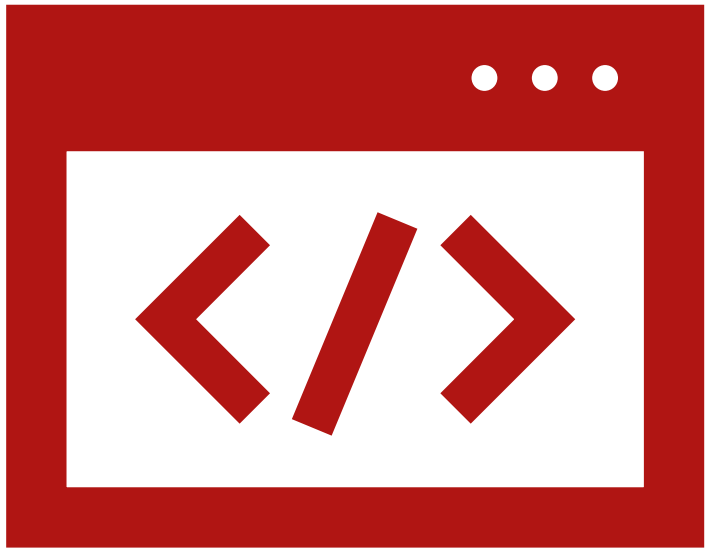
Exemplos da Aula

- https://colab.research.google.com/drive/10dcwlRHu1Vlm1uPzDk8-0SNp_28yghXw?usp=sharing

GEOJSON E GEOCODIFICAÇÃO

FORMATO JSON

- JSON (JavaScript Object Notation) é um formato leve de troca de dados que é fácil de ler e escrever para humanos e fácil para as máquinas analisarem e gerarem. É um formato de texto completamente independente da linguagem, mas que utiliza convenções que são familiares aos programadores da família de linguagens C, incluindo C, C++, C#, Java, JavaScript, Perl, Python, e outros.
- Estrutura :JSON utiliza para organizar dados objetos e arrays.
  - Objetos: são coleções de pares de "key-values", onde as chaves são strings e os valores podem ser qualquer tipo de dado JSON válido (string, número, objeto, array, verdadeiro, falso, ou nulo). Os objetos são rodeados por chaves ({}), e os pares de valores de chaves são separados por vírgulas (,). A chave e o valor são separados por dois pontos (:).
  - Arrays: são coleções ordenadas de valores, que podem ser qualquer tipo de dado JSON válido. As arrays são rodeadas por colchetes ([]), e os valores são separados por vírgulas (,).
- https://www.json.org/json-en.html



JSON

```
{
    "name": "John Doe",
    "age": 32,
    "address": {
        "street": "123 Main St",
        "city": "Anytown",
        "state": "CA"
    },
    "tags": ["Finance", "Admin"]
}
```

Python

```
{
    "name": "John Doe",
    "age": 32,
    "address": {
        "street": "123 Main St",
        "city": "Anytown",
        "state": "CA"
    },
    "tags": ["Finance", "Admin"]
}
```

ScrapingBee

https://favtutor.com/blogs/dict-to-json-python

Conversão JSON ↔ Python

- Para ler JSON:

```
import json
with open('dados.json') as f:
    dados = json.load(f)
```

- Para salvar como JSON:

```
with open('saida.json', 'w') as f:
    json.dump(dados, f)
```

Parsing

- O JSON parsing é o processo de conversão de uma cadeia formatada em JSON em uma estrutura de dados, como um objeto ou array, que pode ser manipulada e usada por um programa. Parsing é necessário porque os dados JSON são armazenados como uma string, mas linguagens de programação requerem tipos de dados estruturados para realizar operações e cálculos.

Erros

- É importante tratar os erros que podem ocorrer durante o JSON parsing. Por exemplo, se a cadeia de entrada não for válida JSON, a análise falhará, e um erro será lançado.

https://geojson.org/

FORMATO GEOJSON

GeoJSON é um formato para codificar uma variedade de estruturas de dados geográficas.

```
{
  "type": "Feature",
  "geometry": {
    "type": "Point",
    "coordinates": [125.6, 10.1]
  },
  "properties": {
    "name": "Dinagat Islands"
  }
}
```

O formato GeoJSON suporta os seguintes tipos de geometria: Point, LineString, Polygon, MultiPoint, MultiLineString e MultiPolygon. Objetos geométricos com propriedades adicionais são objetos de característica. Conjuntos de recursos são contidos por objetos FeatureCollection.

Sistemas de Referência

Na especificação original do GeoJSON (2008), foi incluído suporte para Sistemas de Referência Coordenada (CRS). Entretanto, a partir da especificação atualizada em 2016 (RFC 7946), o suporte para CRS foi removido, e GeoJSON agora assume que todas as coordenadas estão no sistema de referência de coordenadas WGS 84 (World Geodetic System 1984).

De acordo com a especificação atualizada do GeoJSON, o CRS não faz mais parte dos objetos GeoJSON, e todas as coordenadas devem estar em ordem de longitude, latitude (x, y), com valores de elevação (z) opcionais. Isto simplifica o manuseio dos dados GeoJSON e garante a interoperabilidade entre diferentes softwares e ferramentas.

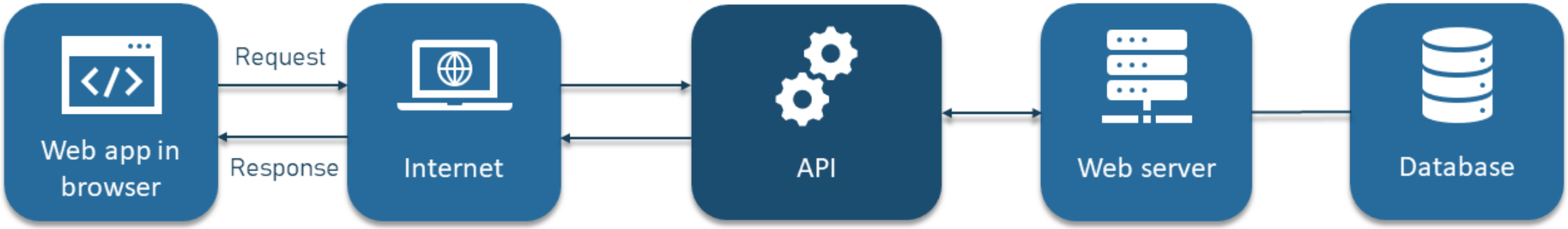
Se você precisar trabalhar com dados em um sistema de referência de coordenadas diferente, você terá que converter as coordenadas para WGS 84 antes de criar um objeto GeoJSON. Você pode usar bibliotecas como pyproj ou GDAL em Python para realizar tais conversões.

Tenha em mente que, de acordo com a especificação GeoJSON, você não deve incluir um objeto CRS em seus dados GeoJSON. Ao invés disso, certifique-se de que todas as coordenadas estejam no formato WGS 84.

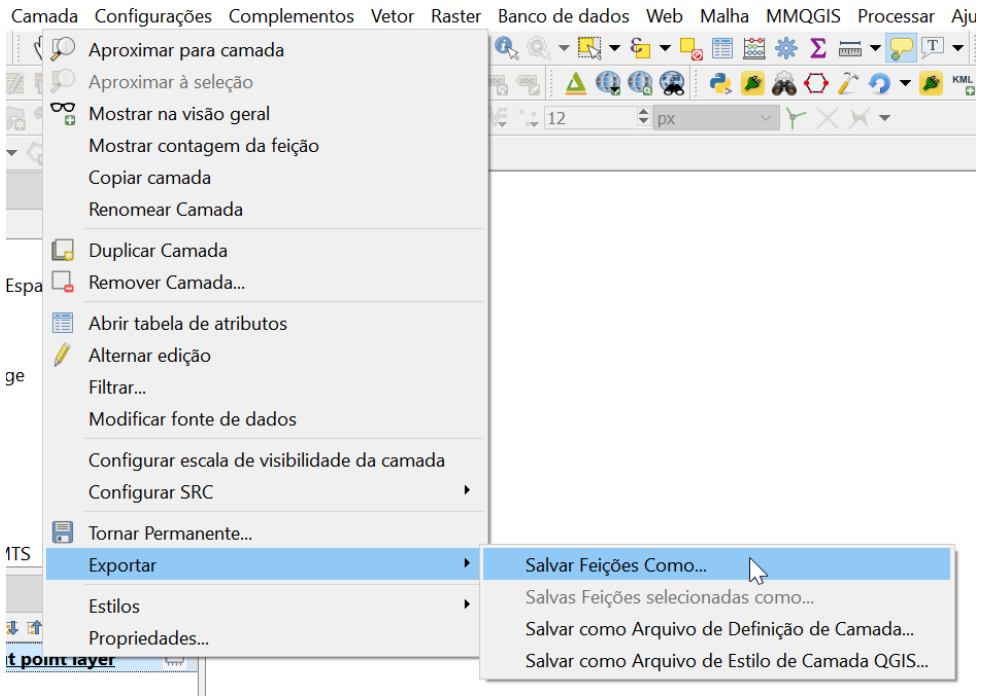
Onde obter GEOJSON

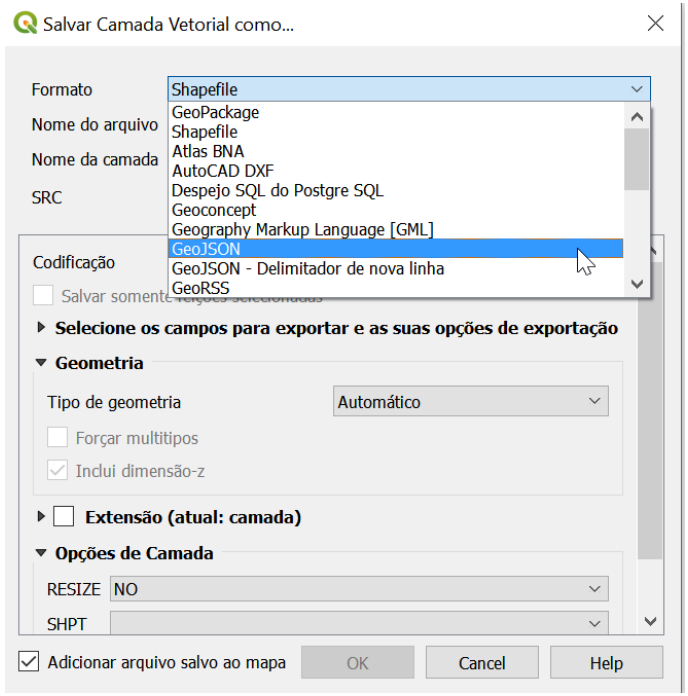
- Conversão de arquivos existentes
- Em um banco de dados, através da função ST_asGEOJSON
- Através de um geoserviço (WFS)
- Através de uma API (por exemplo, Turbopass do OpenStreetMap)

HOW API WORKS



altexsoft





GEOJSON NO QGIS

pandas

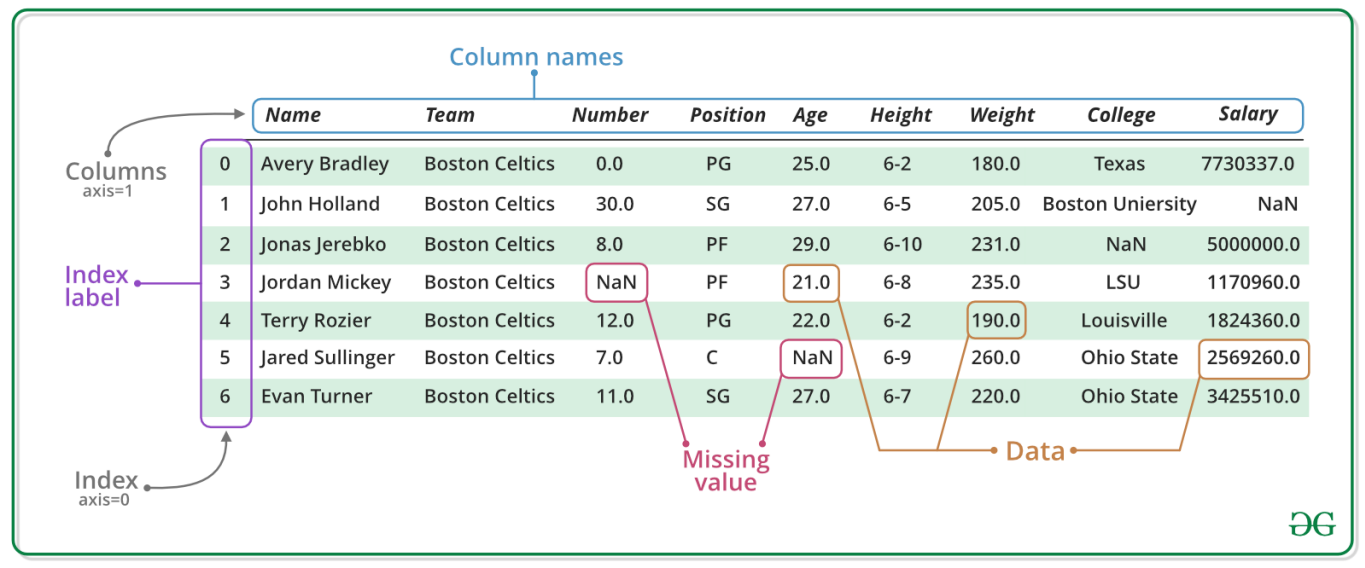
Pandas/Geopandas

Python data analysis (Pandas)

- O Pandas é usado principalmente para análise de dados e manipulação associada de dados tabulares em DataFrames.
- O Pandas permite importar dados de vários formatos de arquivo, como valores separados por vírgula, JSON, Parquet, tabelas ou consultas de banco de dados SQL e Microsoft Excel.
- O Pandas permite várias operações de manipulação de dados, como mesclagem, remodelação, seleção, bem como limpeza de dados.
- O desenvolvimento de pandas introduziu em Python muitos recursos comparáveis de trabalhar com DataFrames que foram estabelecidos na linguagem de programação R.
- A biblioteca pandas é construída sobre outra biblioteca NumPy, que é orientada para trabalhar eficientemente com matrizes em vez dos recursos de trabalhar em DataFrames.

Data Frame (df)

- O que é um DataFrame? Estrutura bidimensional (como uma planilha)



	Name	Team	Number	Position	Age	Height	Weight	College	Salary
0	Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
1	John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston Uniersity	NaN
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0	6-8	235.0	LSU	1170960.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0	6-2	190.0	Louisville	1824360.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN	6-9	260.0	Ohio State	2569260.0
6	Evan Turner	Boston Celtics	11.0	SG	27.0	6-7	220.0	Ohio State	3425510.0

https://fneum.github.io/data-science-for-esm/03-workshop-pandas.html

Funções Básicas - Data Frame

- Lendo dados com Pandas

```
df = pd.read_csv('arquivo.csv')
```

- De um JSON:

```
df = pd.read_json('arquivo.json')
```

- Explorando um DataFrame

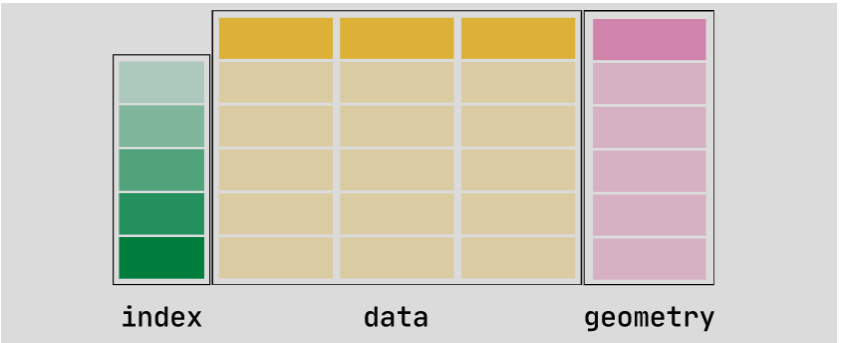
```
df.head()     # Mostra as 5 primeiras linhas
df.tail()     # Últimas 5
df.info()     # Resumo das colunas e tipos
df.describe() # Estatísticas numéricas
```

- Acessando dados no DataFrame

```
df['nome']         # Coluna "nome"
df.loc[0]          # Primeira linha
df.iloc[0, 1]      # Linha 0, coluna 1
```

Geopandas

- Adiciona capacidades espaciais ao Pandas (geoDataFrame)
- GeoPandas estende os tipos de dados usados por pandas para permitir operações espaciais em tipos geométricos. As operações geométricas são realizadas pela biblioteca shapely.
- Geopandas ainda depende da biblioteca fiona para acesso a arquivos e matplotlib para plotagem.



Tipos de Geometrias

- Ponto: Point(x, y)
- Linha: LineString([(x1, y1), (x2, y2), ...])
- Polígono: Polygon([(x1, y1), (x2, y2), ..., (x1, y1)]) # importante fechar

Exemplos:

(SHAPELY)

```
from shapely.geometry import Point, Polygon

ponto = Point(-49.27, -25.43)
poligono = Polygon([(-49.27, -25.43), (-49.26, -25.44), (-49.25, -25.43), (-49.27, -25.43)])
```

(WKT – Well Known Text)

POINT (-49.27 -25.43)

POLYGON ((-49.27 -25.43, -49.26 -25.44, -49.25 -25.43, -49.27 -25.43))

GEOCODIFICAÇÃO VIA GOOGLE MY MAPS

LeafMap

•Wu, Q. (2021). Leafmap: A Python package for interactive mapping and geospatial analysis with minimal coding in a Jupyter environment. *Journal of Open Source Software*, 6(63), 3414. https://doi.org/10.21105/joss.03414

Welcome to leafmap

try lite now | Open Studio Lab | Open Planetary Computer | Open in Colab | launch binder

pypi v0.18.8 | conda-forge v0.18.8 | downloads 145k | docs passing | Linux build passing | License MIT

YouTube Channel | Follow @giswqs | JOSS 10.21105/joss.03414

A Python package for geospatial analysis and interactive mapping in a Jupyter environment.

EXERCÍCIOS

Rampas de cores folium: https://github.com/python-visualization/folium/blob/v0.2.0/folium/utilities.py#L104

Caderno de Exercício 3A (LINK: https://colab.research.google.com/drive/1i__jZLuKqxG8vOxGV2qC-9IXAFIDgMdJ?usp=sharing

Exercício 3B - exiba no mapa os pontos de uma das bases de dados abertos da prefeitura de Curitiba (se necessário filtre para 2000 pontos) https://www.curitiba.pr.gov.br/dadosabertos/

Exercício 3C – https://colab.research.google.com/drive/1Tf8IjITNxDNyGCBwEriyQ7JRVux_xHEp?usp=sharing